

Reviewer Pack — Combinatorial Discrepancy of NW Designs Predicts Generator B...

Entry fc5757929a73 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-26 21:46:21 UTC

Combinatorial Discrepancy of NW Designs Predicts Generator Bias on Hard Truth Tables

Entry ID: fc5757929a73 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-26 21:46:21 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial discrepancy theory of set systems (Beck-Fiala / hereditary discrepancy of the design's incidence matrix) **Field B** (complexity object): Nisan-Wigderson pseudorandom generator output bias against small DNF distinguishers

Statement:

Let $D = \{S_1, \dots, S_m\} \subset [\square]$ be an $(\square, a)$ -NW design with incidence matrix $M \in \{0, 1\}^{m \times \square}$, and let $f: \{0, 1\}^a \rightarrow \{0, 1\}$ be a Boolean function with average sensitivity $AS(f)$. Define the design discrepancy $\text{disc}(D) = \min_{x \in \{\pm 1\}^\square} \|Mx\|_\infty$ and the generator bias $\beta(NW\{D, f\}) = \max \text{ over width-} w \text{ DNFs } T \text{ of } |\Pr_{\{s\}}[T(NW_{\{D, f\}}(s))=1] - \Pr_{\{y\}}[T(y)=1]|$ for $w = \lceil \log m \rceil$. Then for every design D and every f , $\beta(NW_{\{D, f\}}) \leq C \cdot (\text{disc}(D)/m) \cdot 2^{\{AS(f)\}} + 2^{-\Omega(a)}$, with a constant $C \leq 4$ independent of $\square, m, a$, and the bound is tight (within factor 2) on a positive fraction of (D, f) pairs.

Rationale (proposer's reasoning):

The classical NW analysis charges generator bias to the combinatorial intersection structure of the design (sets pairwise meet in $\leq a$ coordinates) but never quantitatively uses the *signed* discrepancy of the incidence matrix, which controls how well a ± 1 seed-coloring can simultaneously balance all output bits. Linking Beck-Fiala / hereditary discrepancy directly to bias would explain why some designs (e.g., random vs Reed-Solomon) yield different fooling constants for the same hard function. The $2^{\{AS(f)\}}$ factor captures how Fourier mass on low-frequency Walsh characters interacts with low-discrepancy seed colorings, a connection essentially absent from the NW literature.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 18aa95790b715802

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥ 200 (D,f) samples spanning $n \in \{6..16\}$, $a \in \{2,3,4\}$, $\beta \leq 16$, $m \leq 2n$ with parity/IP/majority + 5 random f's, conjecture is SUPPORTED iff every sample obeys $\beta \leq 4 \cdot (\text{disc}(D)/m) \cdot 2^{\{AS(f)\}} + 2^{\{-a/2\}}$ AND the lower-envelope slope of β vs $(\text{disc}(D)/m) \cdot 2^{\{AS(f)\}}$ is ≥ 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Nisan-Wigderson generator discrepancy design incidence matrix - pseudo-random generator DNF bias hereditary discrepancy Beck-Fiala - NW design average sensitivity DNF distinguisher pseudorandomness

Top relevant hits considered: - [http://arxiv.org/abs/2212.02103v2] A Study of Hypergraph Using Null Spaces of the Incidence Matrix and its Transpose - [http://arxiv.org/abs/1207.0393v2] Pseudo-finite hard instances for a student-teacher game with a Nisan-Wigderson generator - [http://arxiv.org/abs/1907.01117v2] Exploring Feasible Design Spaces for Heterogeneous Constraints - [http://arxiv.org/abs/2602.09948v1] Non-Additive Discrepancy: Coverage Functions in a Beck-Fiala Setting - [http://arxiv.org/abs/1311.6531v3] Brains and pseudorandom generators - [http://arxiv.org/abs/1306.6081v3] An improvement of the Beck-Fiala theorem - [http://arxiv.org/abs/2412.05161v1] DNF: Unconditional 4D Generation with Dictionary-based Neural Fields - [http://arxiv.org/abs/1201.3684v2] Non-sky-averaged sensitivity curves for space-based gravitational-wave observatories

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.4s

5.1 Generated Python source

```
import random
from itertools import product

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    rank = 0
    for j in range(n):
        i_max = rank
        for i in range(rank, m):
            if abs(A[i][j]) > abs(A[i_max][j]):
                i_max = i
        A[rank], A[i_max] = A[i_max], A[rank]
        if A[rank][j] != 0:
            factor = 1 / A[rank][j]
```

```

        for j2 in range(n):
            A[rank][j2] *= factor
        for i in range(m):
            if i != rank:
                factor = A[i][j]
                for j2 in range(n):
                    A[i][j2] -= factor * A[rank][j2]
            rank += 1
    return rank

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0 for _ in range(p)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][k] += A[i][j] * B[j][k]
    return C

def compute_disc(D, M):
    n = len(M)
    disc = float('inf')
    for coloring in product([-1, 1], repeat=n):
        y = [sum(M[i][j] * coloring[j] for j in range(n)) for i in range(len(D))]
        disc = min(disc, max(abs(y)))
    return disc

def compute_as(f):
    a = len(f)
    count = 0
    for x in product([0, 1], repeat=a):
        for y in product([0, 1], repeat=a):
            if x != y and f(x) == f(y):
                count += 1
    return count / (2 ** a)

def compute_beta(NW_D_f, m):
    n = len(NW_D_f)
    width = (n.bit_length() - 1)
    beta = float('-inf')
    for T in product([0, 1], repeat=n * width):
        T = [T[i:i+width] for i in range(0, n * width, width)]
        pr_s = sum(NW_D_f(s) == 1 for s in product([0, 1], repeat=len(NW_D_f))) / (2 ** len(NW_D_f))
        pr_y = sum(T[y].count(1) / (2 ** y.count(1)) for y in product([0, 1], repeat=n)) / (2 ** n)
        beta = max(beta, abs(pr_s - pr_y))
    return beta

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [6, 8, 11, 14]
    a_values = [2, 3, 4]
    results = []

    for n in n_values:
        for a in a_values:
            m = min(2 * n, 16)

```

```

    k = random.randint(5, 16)

    # Generate an (k, a)-NW design
    D = [set() for _ in range(m)]
    while len(D[0]) < a:
        S = set(random.sample(range(k), a))
        if all(len(S.intersection(T)) <= a for T in D):
            D[random.randint(0, m-1)].update(S)

    # Build the incidence matrix M
    M = [[int(i in S) for i in range(k)] for S in D]

    # Compute disc(D)
    disc = compute_disc(D, M)

    # Define a random Boolean function f
    f = lambda x: random.choice([0, 1])

    # Compute AS(f)
    as_f = compute_as(f)

    # Generate NW_{D,f}
    NW_D_f = [f(tuple(random.choices([0, 1], k=k))) for _ in range(2 ** k)]

    # Compute  $\beta(NW_{D,f})$ 
    beta = compute_beta(NW_D_f, m)

    results.append({
        "metric_name": "beta",
        "metric_value": beta,
        "instances_tested": 1,
        "conjecture_holds": beta <= 4 * (disc / m) * 2 ** as_f + 2 ** (-a / 2),
        "counterexample": ""
    })

mean_beta = sum(result["metric_value"] for result in results) / len(results)
std_beta = (sum((result["metric_value"] - mean_beta) ** 2 for result in results) / len(results)) ** 0.5
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "seed": seed,
    "mean_beta": mean_beta,
    "std_beta": std_beta,
    "support_fraction": support_fraction,
    "results": results
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_beta = sum(result["mean_beta"] for result in result["results"]) / len(result["results"])

```

```

std_beta = (sum((result["mean_beta"] - mean_beta) ** 2 for result in result["results"]) / len
support_fraction = sum(1 for result in result["results"] if result["support_fraction"] >= 0.8)

if support_fraction == 1:
    print(f"RESULT: SUPPORTED mean={mean_beta} std={std_beta} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print("RESULT: INCONCLUSIVE less than 80% seeds support")
else:
    first_failing_seed = next(seed for seed, result in enumerate(result["results"], start=1) if
    print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10778bf3.py", line 136, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10778bf3.py", line 97, in run_trial
    disc = compute_disc(D, M)
            ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10778bf3.py", line 52, in compute_disc
    disc = min(disc, max(abs(y)))
                        ^^^^^^
TypeError: bad operand type for abs(): 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `compute_disc` before any (D,f) samples were produced, so neither the support nor the falsification condition could be evaluated. | next: Fix `compute_disc` to convert the candidate vector `y` to a numeric array (e.g. `np.abs` on `np.array(y)`) and rerun the 200-sample sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	23314
2	preregistration	claude_max	opus	0	0	6156

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	claude_max	opus	0	0	3344
4	novelty	claude_max	opus	0	0	7619
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20020
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17988
7	judge	claude_max	opus	0	0	4160

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 82602 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in *research/audit/fc5757929a73.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/fc5757929a73.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/fc5757929a73.tar.gz* (if generated)